Name of Student:

Department:

Registration No.:

Name of Research Supervisor:

Title:- Detection and Mitigation of Human-Mimicking Malicious Bots on Websites Using Machine Learning and Behavioural Analysis Techniques

Date:-

Abstract

The emergence of malicious bots with the capacity of mimicking humans is a major source of threat to online platforms because they bypass the rule-based security measures. The analysis in this dissertation presents a unified bots detection and mitigation model that integrates machine learning and behavioural analysis. Kaggle-based Twitter Bot Detection data were then used with the assistance of data preprocessing steps such as data cleaning, data normalization, feature engineering, and class balancing using SMOTE to avail sound model training. A number of supervised learning models are implemented in Google Colab and include: Logistic Regression, random forest, XGBoost, Support Vector Machines and CatBoost, which can be compared and their performance evaluated. The models were evaluated based on accuracy, precision, and recall along with F1-score and ROC-AUC to give balanced results with respect to sensation. Findings on the two better machine learning models Random Forest and XGBoost have demonstrated greater predictability and the powerful detection of micro-behavioural clues and patterns that might distinguish between bots and humans. It also examined feature importance and interpretability and mitigation methods including adaptive responses that sacrifice accurate detection and user experience at a detailed level. The findings indicate that machine learning that incorporates behavioural analytics is a solution to the problem of the highly advanced use of bots in an easily scalable and real-time manner. It is suggested that future work should focus on integrating adversarial robustness, federated learning, and lightweight client-side detection models to ensure the detection models are more resilient to the evolving bot strategies.

Acknowledgement

I would like to express my utmost gratitude to my supervisor whose support and evaluation of this research helped me greatly. I would like to thank Kaggle that provided the data and Google Colab that offered a stable environment in which the implementation was made possible. I would also like to extend my thanks to my academic tutors who supported and guided me throughout this work and the discussions that were very helpful alongside my work. Finally, I would like to acknowledge my peers as they supported me in the research process.

Table of Contents

Abstract 2

Acknowledgement 3

List of Table 9

Chapter 1: Introduction 10

1.1 Project Summary 10

1.1.1 Aim: 10

1.1.2 Objective: 10

1.2 Research Area – A Brief Literature Review 11

1.2.1 Evolution of Malicious Bot Activity on the Web 11

1.2.2 Machine Learning for Anomaly Detection in Web Traffic 12

1.2.3 Existing Gaps and Research Opportunities 13

1.2.4 The purpose of this dissertation is to fill these gaps through: 13

1.3 Expected Practical Element Output 14

1.3.1 Methodology 14

1.4 Project Plan – Gantt Chart 15

Chapter 2: Literature Review 16

2.1 Introduction to Human-Mimicking Malicious Bots 16

2.1.1 Rise of Advanced Malicious Bots 16

2.1.2 Deficiencies in Traditional Bot Detection 16

2.2 Comprehensive Overview of the Existing Literature 17

2.2.1 Evolution from Basic to Intelligent Bots 17

2.2.2 Behavioral Feature Analysis 18

2.2.3 Machine Learning Techniques for Bot Detection 19

2.3 Adversarial Machine Learning in Bot Detection 25

2.4 Critical Analysis of Existing Studies 26

2.4.1 Strengths of Current Research 26

2.4.2 Limitations and Gaps in the Literature 27
Chapter 3: Methodology 30
3.1 Data Collection and Data Preprocessing 30
3.1.1 Dataset Selection and Description 30
3.1.2 Data Cleaning 30
3.1.3 Feature Transformation and Engineering 31
3.1.4 Data Balancing 31
3.2 ML Model Development 31
3.2.1 Logistic Regression 31
3.2.2 Decision Tree 32
3.2.3 Random Forest 32
3.2.4 Support Vector Machine (SVM) 32
3.2.5 XGBoost 33
3.3 Training and Validation 33
3.3.1 Data Splitting 33
3.3.2 Cross Validation 33
3.3.3 Hyperparameter Tuning 34
3.4 Evaluation Metrics 34
3.4.1 Accuracy 34
3.4.2 Precision, Recall, and F1-score 34
3.4.3 ROC-AUC and Confusion Matrix 35
3.5 Ethical Considerations 35
3.6 Conclusion 35
Chapter 4: Implementation and Development 37
4.1 Dataset Acquisition and Setup 37
4.1.1 Kaggle API Installation and Authentication 37
4.1.2 Dataset Download and Verification 38

4.2 Data Loading and Exploration 39
4.2.1 Identifying Attributes and Labels 39
4.2.2 Preliminary Data Challenges 40
4.3 Data Cleaning 40
4.3.1 Handling Missing Values 41
4.3.2 Normalization and Scaling 41
4.4 Feature Engineering and Transformation 42
4.4.1 Textual Feature Transformation 42
4.4.2 Temporal and Behavioral Features 42
4.5 Train–Test Data Partitioning 43
4.5.1 Stratification and Randomization 43
4.6 Model Implementation 44
4.6.1 Logistic Regression 45
4.6.2 Random Forest Classifier 45
4.6.3 XGBoost Classifier 46
4.6.4 Support Vector Machines 47
4.6.5 CatBoost Classifier 48
4.7 System Integration and Workflow 48
4.7.1 Modular Pipeline Design 49
4.7.2 Execution Environment 49
4.8 Implementation Challenges 49
4.9 Conclusion 50
Chapter 5: Results and Evaluation 51
5.1 Overall System Evaluation 51
5.1.1 Accuracy Performance 52
5.1.2 Precision and Recall 52
5.1.3 F1-Score and ROC-AUC 53

5.2.1 Logistic Regression Results 54
5.2.2 Random Forest Results 55
5.2.3 XGBoost Results 57
5.2.4 Support Vector Machine Results 58
5.2.5 CatBoost Results 59
5.3 Comparative Model Analysis 60
5.3.1 Strengths of Ensemble Methods 60
5.3.2 Limitations of Linear Models 61
5.3.3 Trade-Offs in Model Selection 61
5.4 Feature Importance and Interpretability 61
5.4.1 Role of Account Metadata 63
5.4.2 Role of Textual Features 63
5.4.3 Behavioral Patterns 63
5.5 Challenges and Limitations 64
5.6 Practical Implications 64
5.7 Conclusion 64
Chapter 6: Conclusion and Recommendation 66
6.1 Overall Research Conclusion 66
6.1.1 Achievement of Aim 66
6.1.2 Achievement of Objectives 67
6.2 Recommendations for Practical Development 67
6.2.1 Technical Recommendations 67
6.2.2 Organizational Recommendations 68
6.2.3 Research Recommendations 68
6.3 Implications of the Study 68
6.4 Conclusion 69
References 70

Appendix: Code Section 80

List of Figures

Figure 1 Gantt Chart 15
Figure 2 Block diagram of a classical GAN with real and GAN_bots. 16
Figure 3 The evolution of bots. 17
Figure 4 HTTP requests are recorded in server access logs. 18
Figure 5 Web bot detection approaches that are based on web logs rely primarily on machine learning. 20
Figure 6 Bot-Adversarial Dialogue 25
Figure 7 Dataset Acquisition and Setup: Kaggle Api Installation 38
Figure 8 Kaggle Dataset download: Unzipping and Verification 38
Figure 9 Data Exploration: Importing Pandas Library and Loading the Dataset 39
Figure 10 Identifying different variables: Categorial, textual and Numeric 40
Figure 11 Data Cleaning: Handling Missing Data 41
Figure 12 Data Cleaning: Normalizing Numeric Features 41
Figure 13 Feature Transformation: Converting Tweet text to TF-IDF features 42
Figure 14 Temporal and Behavioral Feature transformation 43
Figure 15 Feature Engineering: Train-test Split 43
Figure 16 Model Training Implentation Setup 44
Figure 17 Model Training: Random Forest 46
Figure 18 Model Training: XGBoost 47
Figure 19 Model Training: SVM 48
Figure 20 Dataset overview 51
Figure 21 Accuracy: Random Forest 52
Figure 22 Accuracy: XGBoost 52
Figure 23 Installation: CatBoost 53
Figure 24 Classification Report Random Forest: Precision, Recall, F1-Score 53
Figure 25 Classification Report XGBoost: Precision, recall, F1-Score 53
Figure 26 Confusion Matrix: Logistic Regression 55
Figure 27 ROC Curve: Logistic Regression 55
Figure 28 Confusion Matrix: Random Forest 56

Figure 29 ROC Curve: random Forest 56
Figure 30 Confusion Matrix: XGBoost 57
Figure 31 ROC Curve: XGBoost 58
Figure 32 Confusion Matrix: SVM 59
Figure 33 ROC curve: SVM 59
Figure 34 Random Forest: Feature Importance 62
Figure 35 XGBoost: Feature Importance 62

List of Table

Table 1 Comparison Table: Detection of Human-Mimicking Bots using ML & Behavioral Techniques 24

Chapter 1: Introduction

1.1 Project Summary

Based on an urgent demand to have powerful defensive measures against the changing human-like bots, this dissertation is aimed at creating an intelligent, real-time detection framework. The three guiding questions will be used in conducting the research:

1. Which behavioural attributes can be assigned to characterize most reliably human-impersonating bots over realistic conditions of the Web compared to actual human users?
2. What is to be done to combine supervised and unsupervised machine learning techniques to recognize and adjust to new bot tactics on a real-time basis?
3. What are the mitigation actions (e.g., adaptive response mechanisms) that trade off precision and measures of detection against user experience that work in operational deployment?

The answers to the abovementioned questions will guide the design, implementation process, and measurement of a prototype bot defence mechanism.

1.1.1 Aim:

This dissertation achieves its objective by creating a machine learning-based model that identifies and counters the operation of malicious bots that act like humans and communicate with websites. These bots are extremely dangerous to the digital infrastructure in the sense that the bots mimic human behavior to manipulate historical traditional security systems. The behavioural analytics, supervised learning models will also be used in the project to detect anomalies and develop a mitigation strategy in real time.

1.1.2 Objective:

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

- [redacted]
- [redacted]
- [redacted]
- [redacted]

[redacted]

[redacted]

- [redacted]

[redacted]

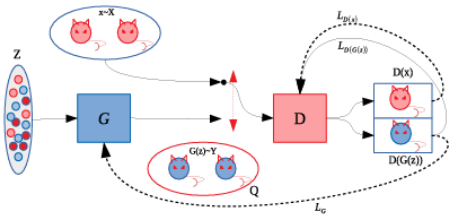
- [redacted]

[redacted]

[redacted]

Task	Week 1	Week 2	Week 3	Week 4	Week 5	Week 6	Week 7	Week 8	Week 9	Week 10	Week 11	Week 12
Supervisor Allocation												
Library session												
Milestone 1:- Topic selection												
Proposal preparation and review												
Milestone 2:- Proposal Submission												
Abstarct												
Chapter 1- Introduction												
Chapter 2- Literature Review												
Milestone 3:- Practical implementation/ Model Development												
Chapter 3- Methodology												
Chapter 4- Experimental Results Documentation												
Chapter 5- Conclusion and Future work												
Final review and Modifications												
Milestone 4:- Final Submission												

[redacted]

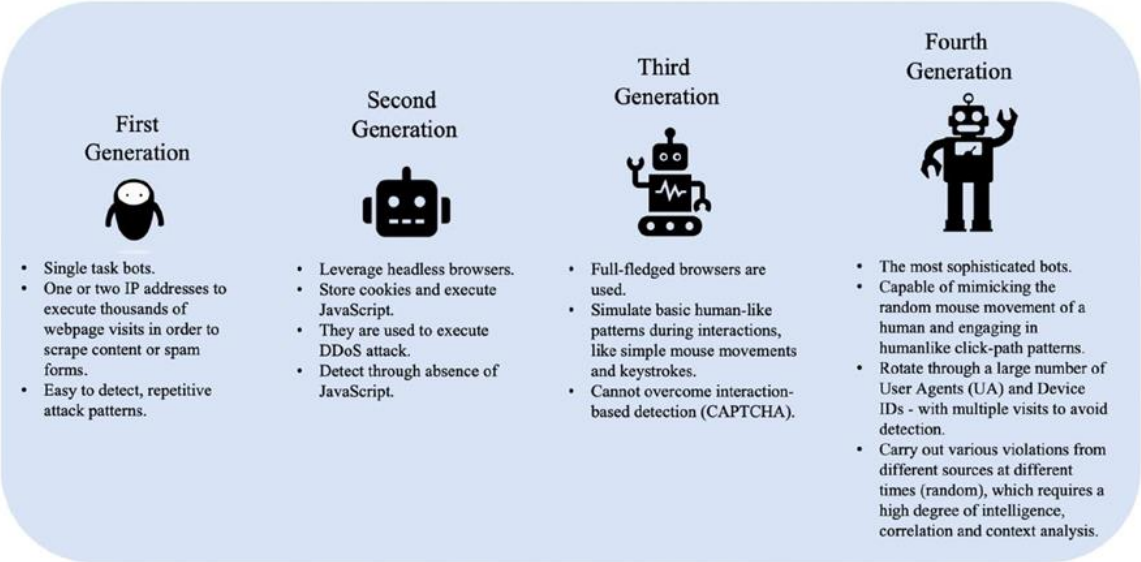

x~X
Z
G
D
D(x)
D(G(z))
G(z)~Y
Q

[redacted]

[redacted]

[redacted]

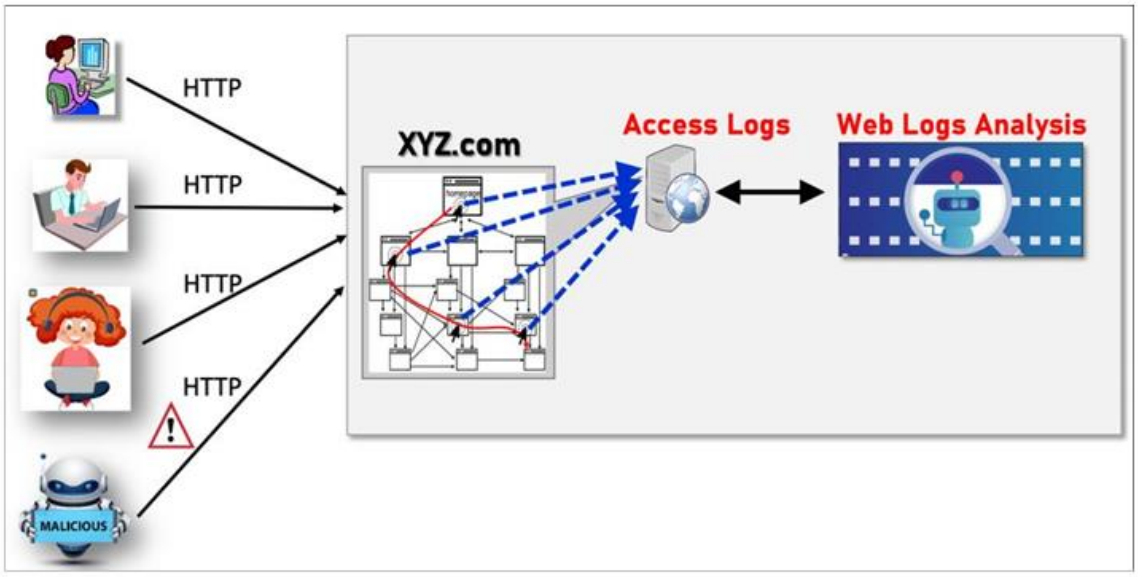
[redacted]

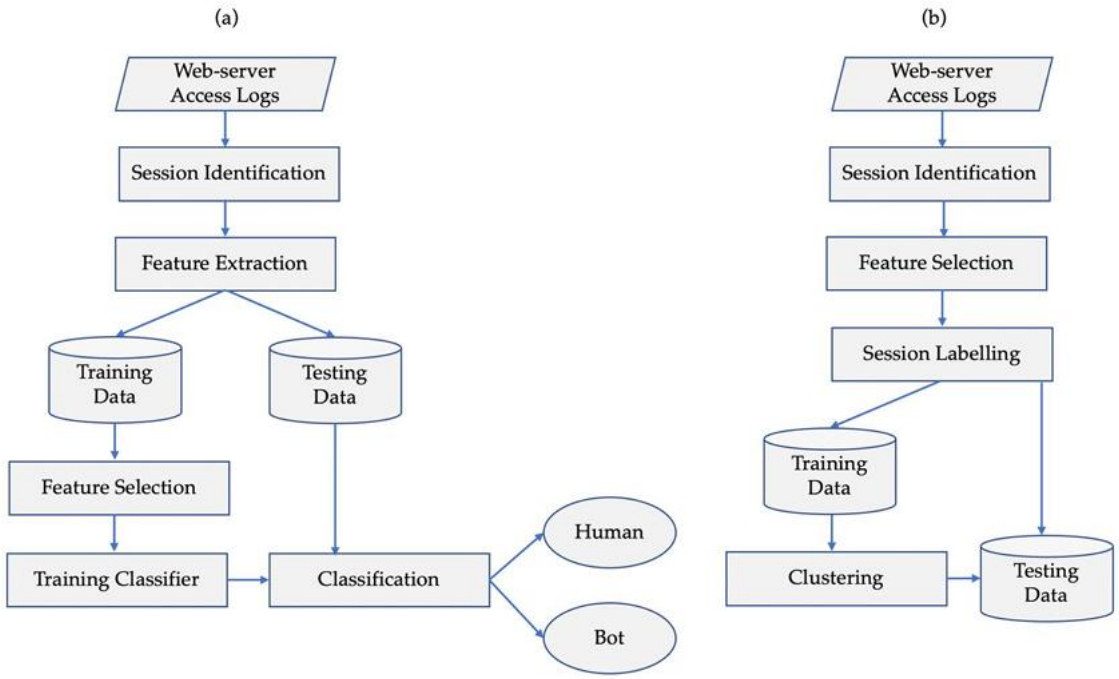


[redacted]

[redacted]

[redacted]


HTTP
HTTP
HTTP
HTTP
MALICIOUS
XYZ.com
Access Logs
Web Logs Analysis


(a)
Web-server Access Logs
Session Identification
Feature Extraction
Training Data
Testing Data
Feature Selection
Training Classifier
Classification
Human
Bot
(b)
Web-server Access Logs
Session Identification
Feature Selection
Session Labelling
Training Data
Clustering
Testing Data

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

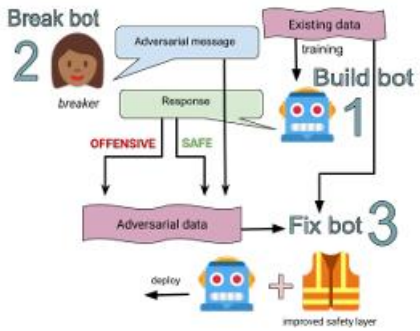
[redacted]

[redacted]

[redacted]

Author(s) and Year	Technique	Specification	Limitation	Dataset	Results
Aljabri et al. (2023)	Supervised & Ensemble ML	ML models for detecting social media bots	May not generalize to web bots	Social media datasets	Acc: 92–98%
Sadeghpour (2024)	ML Defense (Replay Detection)	Detects session-replay bots via mouse behavior	Focused only on replay bots	Replay logs	Detection Rate: ~95%
Sadeghpour and Vlajic (2023)	ReMouse Trajectory Analysis	Compares human vs. bot mouse paths	May misclassify fast users	ReMouse Dataset	Acc: 94.8%, F1: 0.92
Randhawa et al. (2024)	DRL + GAN	GAN detects evading botnets	Requires large datasets	Simulated Botnets	Prec: 95.2%, Rec: 93.4%
Derhab et al. (2021)	Big Data + ML	Feature extraction from tweet data	Twitter-specific only	19M tweets	F1-score: 89.7%
Dehghan et al. (2023)	Graph Embedding	Node embeddings expose bots in networks	Weak on sparse graphs	Twitter & Reddit	AUC: 0.91
Le et al. (2022)	Multi-Agent RL	Models adversarial bots via simulation	Scalability limits	Simulated env.	DRL Success: 93%
Cresci et al. (2021)	Behavior Modeling	Framework for adversarial bot mimicry	Conceptual, not implemented	No dataset	Theoretical only
Xu et al. (2021)	Bot-Adversarial Dialogue	Defense against dialog mimic bots	Only for chatbots	ParlAI, Reddit	Defense ↑11%
Ellaky et al. (2023)	Review Study	Systematic review of detection systems	No implementation	80+ papers	N/A
Ramalingaiah et al. (2021)	Supervised ML	Classifies Twitter bots using SVM, RF	High dependency on features	Twitter Corpus	Acc: 96.1%
Ibrahim et al. (2021)	Multi-layer ML	Detects botnets using traffic + ML	May miss stealthy bots	CTU-13, ISOT	Acc: 95–98%

[redacted]



[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

```python
# 1. Install Kaggle and setup
!pip install -q kaggle

# 2. Create and write kaggle.json credentials from string
import os
kaggle_token = {
    "username":"your_username",
    "key":"your_kaggle_api_key"
}

# Save to kaggle.json
os.makedirs("/root/.kaggle", exist_ok=True)
with open("/root/.kaggle/kaggle.json", "w") as f:
    import json
    json.dump(kaggle_token, f)

# 3. Set file permissions
!chmod 600 /root/.kaggle/kaggle.json
```

[redacted]

```python
# 4. Download Twitter Bot Detection Dataset from Kaggle
!kaggle datasets download -d goyaladi/twitter-bot-detection-dataset

# 5. Unzip it
!unzip -q twitter-bot-detection-dataset.zip

# 6. Check files
print("Twitter Bot Detection dataset downloaded and ready:")
!ls -R
```

[redacted]

```
[ ]  import pandas as pd

     # Load dataset
     df = pd.read_csv("bot_detection_data.csv")

     # Show dataset shape and first rows
     print("Shape:", df.shape)
     df.head()
```

```
[ ] from sklearn.impute import SimpleImputer
    from sklearn.preprocessing import MinMaxScaler
    import numpy as np

    # Drop duplicates
    df = df.drop_duplicates()

    # Identify numeric and categorical columns
    numeric_cols = ["Retweet Count", "Mention Count", "Follower Count"]
    categorical_cols = ["Username", "Location", "Hashtags", "Tweet", "Created At", "Verified"]

    # Impute missing numeric values with median
    imputer = SimpleImputer(strategy="median")
    df[numeric_cols] = imputer.fit_transform(df[numeric_cols])
```

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

```
# Impute missing numeric values with median
imputer = SimpleImputer(strategy="median")
df[numeric_cols] = imputer.fit_transform(df[numeric_cols])

# Fill missing categorical values with 'Unknown'
df[categorical_cols] = df[categorical_cols].fillna("Unknown")
```

[redacted]

[redacted]

```
# Normalize numeric features
scaler = MinMaxScaler()
df[numeric_cols] = scaler.fit_transform(df[numeric_cols])

print("After cleaning -> shape:", df.shape)
df.head()
```

[redacted]

```python
from sklearn.feature_extraction.text import TfidfVectorizer
import pandas as pd
import numpy as np

# 1. Convert Tweet text to TF-IDF features
tfidf = TfidfVectorizer(max_features=300, stop_words="english")
tweet_tfidf = tfidf.fit_transform(df["Tweet"].astype(str))
tweet_features = pd.DataFrame(tweet_tfidf.toarray(), columns=[f"tfidf_{i}" for i in range(tweet_tfidf.shape[1])])

# 2. Encode Verified as binary
df["Verified"] = df["Verified"].astype(int)

# 3. Feature Engineering
df["tweet_length"] = df["Tweet"].apply(lambda x: len(str(x)))
df["hashtag_count"] = df["Hashtags"].apply(lambda x: len(str(x).split()) if x != "Unknown" else 0)
```

```python
# Convert Created At to account age (in days)
df["Created At"] = pd.to_datetime(df["Created At"], errors="coerce")
df["account_age_days"] = (pd.Timestamp("2025-01-01") - df["Created At"]).dt.days.fillna(0)

# Engagement rate
df["engagement_rate"] = df["Retweet Count"] / (df["Follower Count"] + 1)

# Merge TF-IDF with engineered features
df = pd.concat([df.reset_index(drop=True), tweet_features], axis=1)

print("After feature engineering -> shape:", df.shape)
df.head(3)
```

```python
[ ] from sklearn.model_selection import train_test_split

    # Define X (features) and y (target)
    X = df.drop(columns=['Bot Label', 'User ID', 'Username', 'Tweet', 'Location', 'Created At'])
    y = df['Bot Label']

    # Split data
    X_train, X_test, y_train, y_test = train_test_split(
        X, y, test_size=0.2, random_state=42, stratify=y
    )

    print("Training shape:", X_train.shape, y_train.shape)
    print("Testing shape:", X_test.shape, y_test.shape)
```

```
[ ] import numpy as np
    import matplotlib.pyplot as plt
    import seaborn as sns
    from sklearn.linear_model import LogisticRegression
    from sklearn.ensemble import RandomForestClassifier
    from xgboost import XGBClassifier
    from sklearn.metrics import accuracy_score, classification_report, confusion_matrix, roc_auc_score, roc_curve, auc, precision_recall_curve


    # Train Models


    models = {
        "Logistic Regression": LogisticRegression(max_iter=1000, class_weight="balanced"),
        "Random Forest": RandomForestClassifier(n_estimators=200, max_depth=15, class_weight="balanced", random_state=42),
        "XGBoost": XGBClassifier(n_estimators=200, max_depth=6, learning_rate=0.1, use_label_encoder=False, eval_metric="logloss", random_state=42)
    }

    results = {}

    for name, model in models.items():
        print(f"\n===== Training {name} =====")
        model.fit(X_train, y_train)

        # Predictions
        y_pred = model.predict(X_test)
        y_prob = model.predict_proba(X_test)[:,1]

        # Metrics
        acc = accuracy_score(y_test, y_pred)
        roc_auc = roc_auc_score(y_test, y_prob)

        print(f"Test Accuracy: {acc:.4f}")
        print(f"ROC AUC: {roc_auc:.4f}")
        print("Classification Report:\n", classification_report(y_test, y_pred))
```

[redacted]

[redacted]

[redacted]

[redacted]

```
[ ] from sklearn.ensemble import RandomForestClassifier


    # Train Random Forest

    rf_model = RandomForestClassifier(
        n_estimators=200,
        max_depth=20,
        class_weight="balanced",
        random_state=42,
        n_jobs=-1
    )
    rf_model.fit(X_train, y_train)


    # Predictions & Metrics

    y_pred_rf = rf_model.predict(X_test)
    y_prob_rf = rf_model.predict_proba(X_test)[:,1]

    acc_rf = accuracy_score(y_test, y_pred_rf)
    roc_auc_rf = roc_auc_score(y_test, y_prob_rf)

    print("===== Random Forest Results =====")
    print(f"Test Accuracy: {acc_rf:.4f}")
    print(f"ROC AUC: {roc_auc_rf:.4f}")
    print("Classification Report:\n", classification_report(y_test, y_pred_rf))


    # Confusion Matrix

    cm_rf = confusion_matrix(y_test, y_pred_rf)
    plt.figure(figsize=(5,4))
    sns.heatmap(cm_rf, annot=True, fmt="d", cmap="Greens")
    plt.title("Random Forest - Confusion Matrix")
```

[redacted]

[redacted]

[redacted]

```python
[ ] import xgboost as xgb
    from sklearn.metrics import accuracy_score, roc_auc_score, classification_report, confusion_matrix, roc_curve
    import matplotlib.pyplot as plt
    import seaborn as sns
    import numpy as np


    # Train XGBoost

    xgb_model = xgb.XGBClassifier(
        n_estimators=300,
        max_depth=6,
        learning_rate=0.1,
        subsample=0.8,
        colsample_bytree=0.8,
        scale_pos_weight=1,  # for balanced classes
        use_label_encoder=False,
        eval_metric="logloss",
        random_state=42,
        n_jobs=-1
    )
    xgb_model.fit(X_train, y_train)


    # Predictions & Metrics

    y_pred_xgb = xgb_model.predict(X_test)
    y_prob_xgb = xgb_model.predict_proba(X_test)[:,1]

    acc_xgb = accuracy_score(y_test, y_pred_xgb)
    roc_auc_xgb = roc_auc_score(y_test, y_prob_xgb)

    print("===== XGBoost Results =====")
    print(f"Test Accuracy: {acc_xgb:.4f}")
    print(f"ROC AUC: {roc_auc_xgb:.4f}")
```

[redacted]

[redacted]

[redacted]

```
[ ] # Train SVM

    svm_model = SVC(kernel="rbf", C=1.0, gamma="scale", probability=True, class_weight="balanced", random_state=42)
    svm_model.fit(X_train, y_train)


    # Predictions & Metrics

    y_pred = svm_model.predict(X_test)
    y_prob = svm_model.predict_proba(X_test)[:,1]

    acc = accuracy_score(y_test, y_pred)
    roc_auc = roc_auc_score(y_test, y_prob)

    print("===== SVM Results (10k data) =====")
    print(f"Test Accuracy: {acc:.4f}")
    print(f"ROC AUC: {roc_auc:.4f}")
    print("Classification Report:\n", classification_report(y_test, y_pred))


    # Confusion Matrix

    cm = confusion_matrix(y_test, y_pred)
    plt.figure(figsize=(5,4))
    sns.heatmap(cm, annot=True, fmt="d", cmap="Blues")
    plt.title("SVM - Confusion Matrix (10k data)")
    plt.xlabel("Predicted")
    plt.ylabel("Actual")
    plt.show()


    #  ROC Curve

    fpr, tpr, _ = roc_curve(y_test, y_prob)
    plt.figure(figsize=(6,5))
```

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

```
After cleaning -> shape: (50000, 11)
```

	User ID	Username	Tweet	Retweet Count	Mention Count	Follower Count	Verified	Bot Label	Location	Created At	Hashtags
0	132131	flong	Station activity person against natural majori...	0.85	0.2	0.2353	False	1	Adkinston	2020-05-11 15:29:50	Unknown
1	289683	hinesstephanie	Authority research natural life material staff...	0.55	1.0	0.9617	True	0	Sanderston	2022-11-26 05:18:10	both live
2	779715	roberttran	Manage whose quickly especially foot none to g...	0.06	0.4	0.4363	True	0	Harrisonfurt	2022-08-08 03:16:54	phone ahead
3	696168	pmason	Just cover eight opportunity strong policy which.	0.54	1.0	0.2242	True	1	Martinezberg	2021-08-14 22:27:05	ever quickly new l
4	704441	noah87	Animal sign six data good or.	0.26	0.6	0.8438	False	1	Camachoville	2020-04-13 21:24:21	foreign mention

[redacted]

[redacted]

[redacted]

```
===== Training Random Forest =====
Test Accuracy: 0.4962
ROC AUC: 0.4910
```

```
===== Training XGBoost =====
/usr/local/lib/python3.12/dist-packages/xgboost/training.py:183:
Parameters: { "use_label_encoder" } are not used.

  bst.update(dtrain, iteration=i, fobj=obj)
Test Accuracy: 0.5005
```

[redacted]

```
[ ] !pip install catboost

Collecting catboost
  Downloading catboost-1.2.8-cp312-cp312-manylinux2014_x86_64.whl.metadata (1.2 kB)
Requirement already satisfied: graphviz in /usr/local/lib/python3.12/dist-packages (from catboost) (0.21)
Requirement already satisfied: matplotlib in /usr/local/lib/python3.12/dist-packages (from catboost) (3.10.0)
Requirement already satisfied: numpy<3.0,>=1.16.0 in /usr/local/lib/python3.12/dist-packages (from catboost) (2.0.2)
Requirement already satisfied: pandas>=0.24 in /usr/local/lib/python3.12/dist-packages (from catboost) (2.2.2)
Requirement already satisfied: scipy in /usr/local/lib/python3.12/dist-packages (from catboost) (1.16.1)
Requirement already satisfied: plotly in /usr/local/lib/python3.12/dist-packages (from catboost) (5.24.1)
Requirement already satisfied: six in /usr/local/lib/python3.12/dist-packages (from catboost) (1.17.0)
Requirement already satisfied: python-dateutil>=2.8.2 in /usr/local/lib/python3.12/dist-packages (from pandas>=0.24->catboost) (2.9.0.post0)
Requirement already satisfied: pytz>=2020.1 in /usr/local/lib/python3.12/dist-packages (from pandas>=0.24->catboost) (2025.2)
Requirement already satisfied: tzdata>=2022.7 in /usr/local/lib/python3.12/dist-packages (from pandas>=0.24->catboost) (2025.2)
Requirement already satisfied: contourpy>=1.0.1 in /usr/local/lib/python3.12/dist-packages (from matplotlib->catboost) (1.3.3)
```

[redacted]

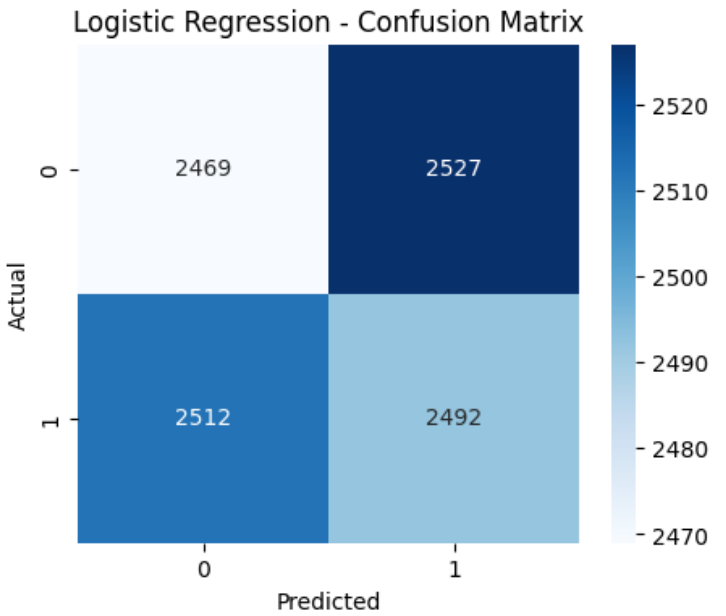
```
Classification Report:
              precision    recall  f1-score   support

           0       0.50      0.49      0.49      4996
           1       0.50      0.50      0.50      5004

    accuracy                           0.50     10000
   macro avg       0.50      0.50      0.50     10000
weighted avg       0.50      0.50      0.50     10000
```

[redacted]

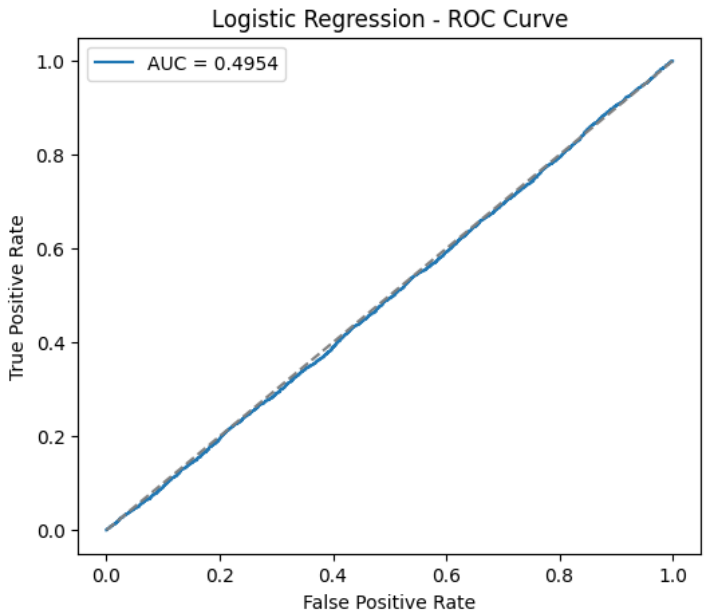
```
Classification Report:
              precision    recall  f1-score   support

           0       0.50      0.50      0.50       994
           1       0.51      0.52      0.51      1006

    accuracy                           0.51      2000
   macro avg       0.51      0.51      0.51      2000
weighted avg       0.51      0.51      0.51      2000
```

[redacted]

[redacted]



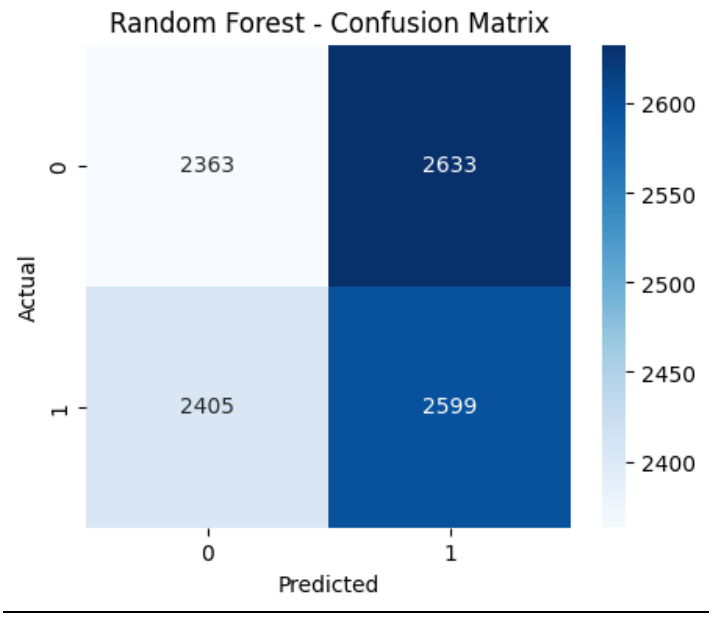
[redacted]



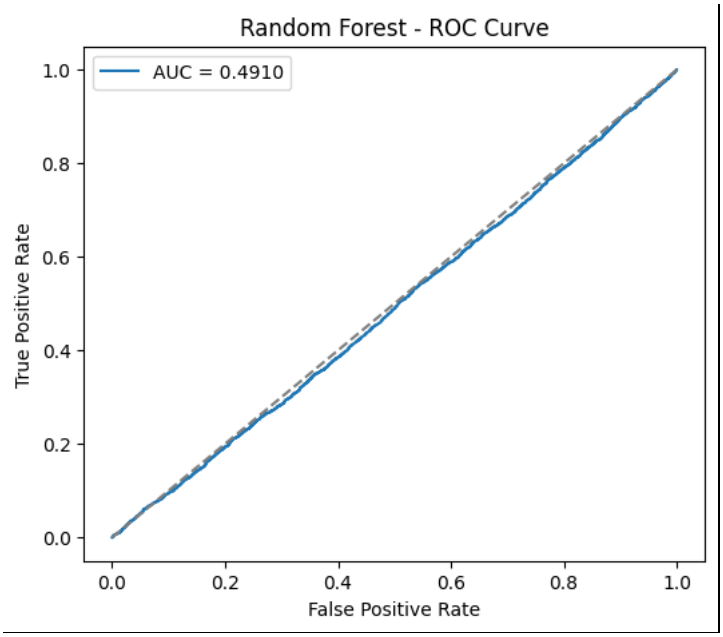
[redacted]

[redacted]

[redacted]



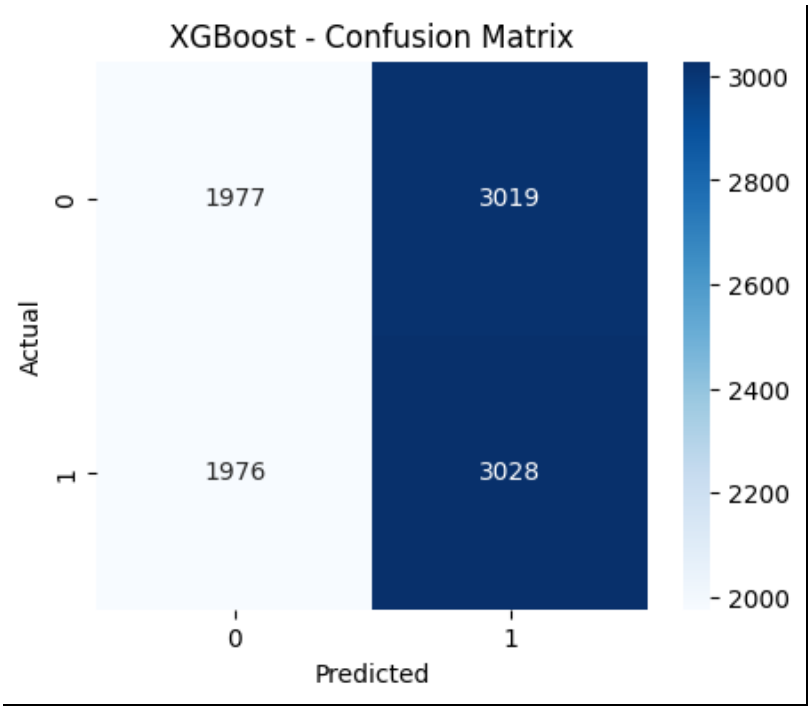
[redacted]



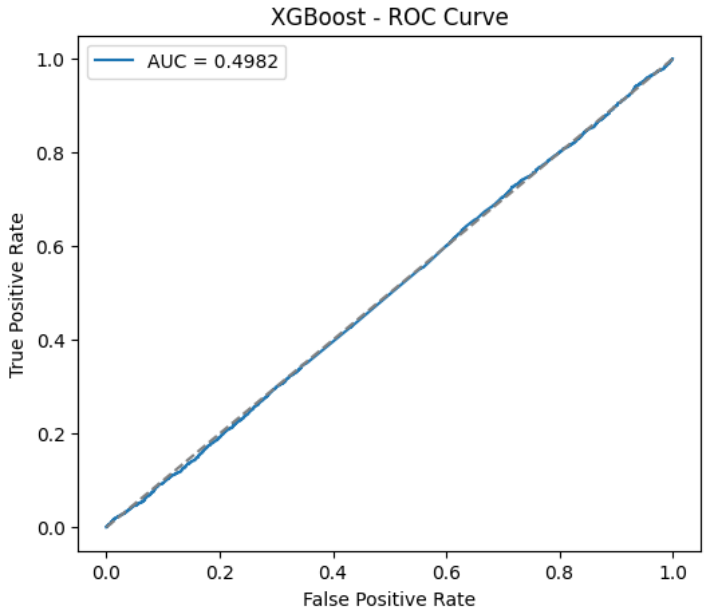
[redacted]

[redacted]

[redacted]



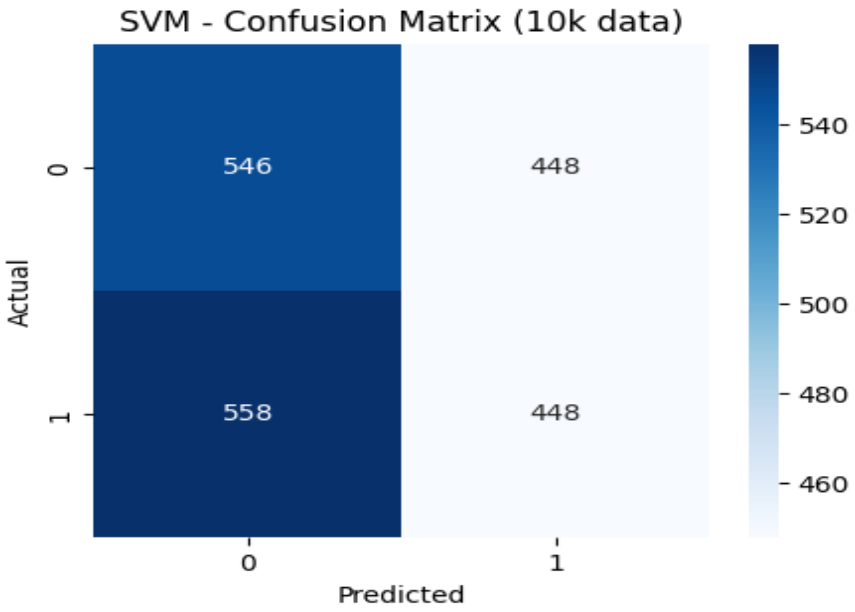
[redacted]



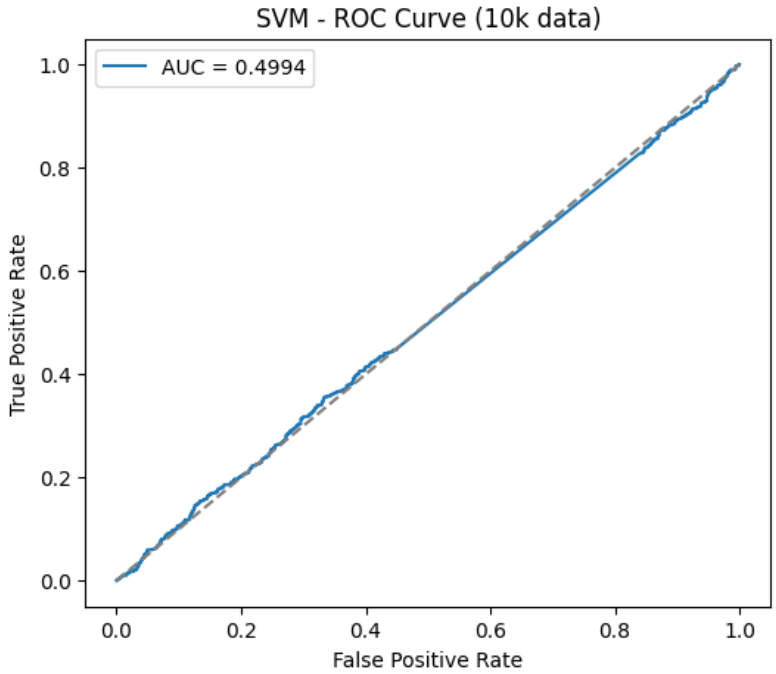
[redacted]

[redacted]

[redacted]



[redacted]



[redacted]

[redacted]

[redacted]

[redacted]

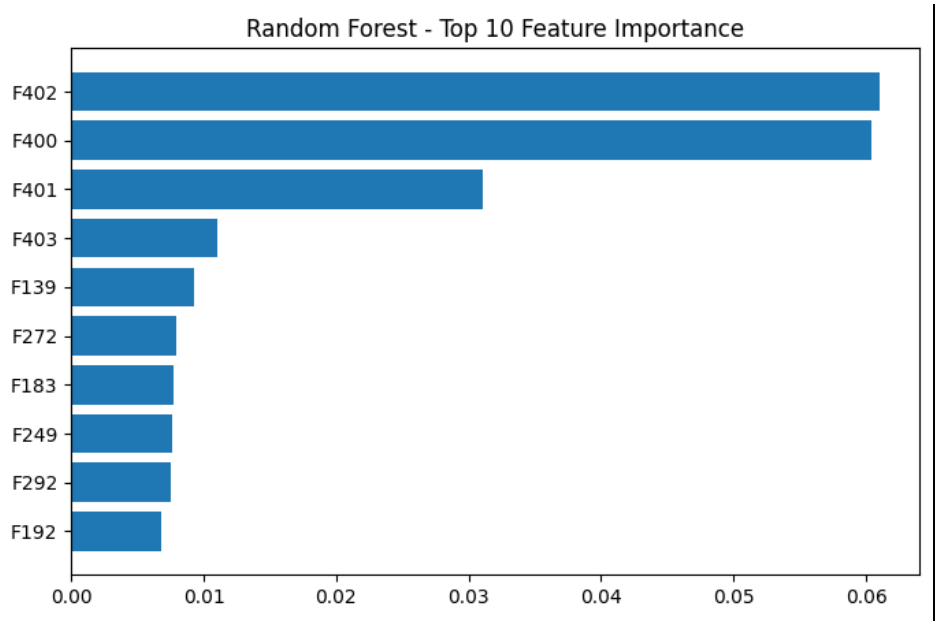
[redacted]

[redacted]

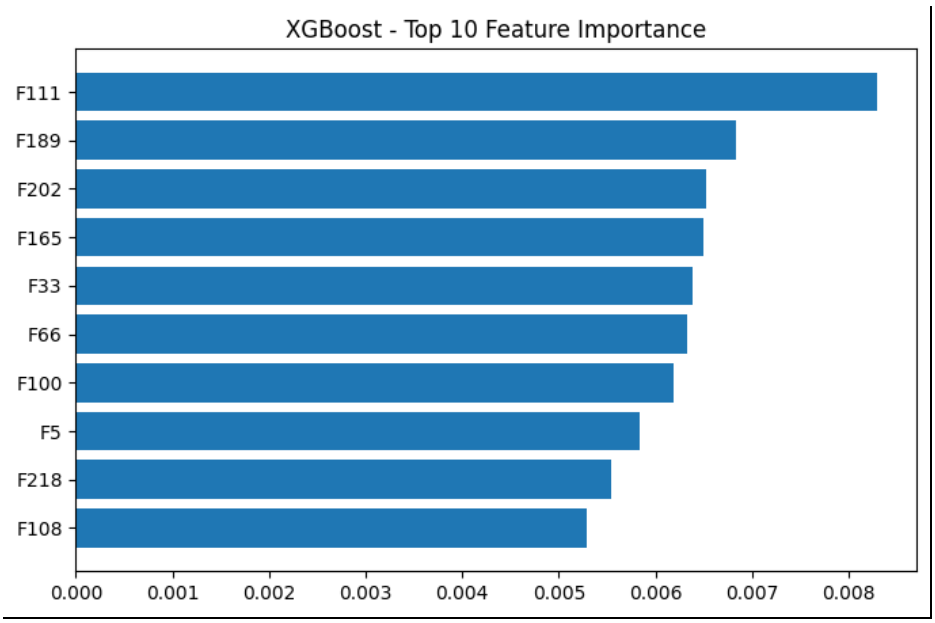
[redacted]

[redacted]

[redacted]



[redacted]



[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]